

2026 《人工智能导论》大作业

任务名称： 可解释的谣言预测

完成组号： 2

小组人员： 雷双豪（组长）、李哲（B）、张
钧策(C)、陈志恒(D)

完成时间： 2026.6.17

1. 任务目标

基于课程提供的推文数据集（`train.csv` / `val.csv`），构建一个可解释的谣言检测系统。系统输入一条英文推文，需同时输出两个结果：

输出 1（分类）：二分类标签，0 = 非谣言，1 = 谣言，分类准确率越高越好；

输出 2（解释）：一段中文判断依据，解释为何判定为谣言或非谣言。

2. 具体内容

（1）实施方案

成员分工

雷双豪（组长）：编写 `inference.py`、`data_clean.py`，完成推理功能和清洗数据、统一接口、维护仓库、撰写报告

李哲（B）：在 `src` 下完成 `/train_baseline.py`、`train_bigru.py`、`train_bert.py`，保存模型到 `/models`，完成模型训练和参数保存

张钧策（C）：完成 LLM 调用部分，在 `src/` 下完成 `llm_api.py`、`prompts.py`、`explain.py`，设计 `prompt`，生成判断依据

陈志恒（D）：模型评估，编写 `src/evaluate.py`，在验证集 `val_clean.csv` 上评估，分析模型效果、完成混淆矩阵和柱状图

系统采用 分类模型 + LLM 解释 的复合架构，分为四个模块：

数据清洗：原始推文包含 HTML 实体、URL、@提及、#话题标签等噪声。清洗流程：HTML 实体解码 → URL 移除 → @提及统一替换为 '@USER' → #标签保留文字去除 '#' 符号 → 小写 → 去标点。同时执行标签冲突移除、完全/近似去重、训练/验证集重叠移除。清洗后训练集 2,779 条（非谣言 1,565 / 谣言 1,214），验证集 400 条（非谣言 226 / 谣言 174）

分类模型：实现两种分类模型，覆盖传统机器学习与深度学习

TF-IDF + 逻辑回归（LR）：FeatureUnion 融合词级，逻辑回归分类

BiGRU：词嵌入+全连接输出，二分类（sigmoid）

BERT 微调（bert-base-uncased）作为可选加分模型已完成训练，但未纳入默认评估。

解释生成：调用 SJTU LLM API（DeepSeek 模型， temperature=0.3 , max_tokens=256）生成中文判断依据。设计三种 Prompt 策略：

1.v0：直接提问，给出检测结果后询问为什么

2.v1（默认）：思维链引导，针对谣言/非谣言分别分析步骤（来源、措辞、逻辑等维度）；

3.fewshot：提供 3 个带解释的标注案例作为上下文参考。

LLM 不可用时自动回退到规则模板：基于关键词匹配（如 'BREAKING'、'anonymous'、'shocking' 等暗示谣言；'according to'、'official'、'confirmed' 等暗示非谣言）生成解释。

评估与集成：评估模块在验证集上计算准确率、精确率、召回率、F1（宏平均 + 谣言类单独），输出混淆矩阵图、对比柱状图、错误样本。推理模块提供统一命令行入口 ``python src/inference.py``，默认加载 BiGRU 模型，支持 ``--model`` 切换，支持交互模式。

（2）核心代码分析

数据清洗 (`data_clean.py`):

``clean_text()`` 函数封装完整的清洗流水线，被训练、评估、推理模块复用。去重算法采用 fuzzy matching (``SequenceMatcher`` + 滑动窗口)，阈值为 0.85。

BiGRU 训练(`train_bigru.py`)

模型结构：``Embedding(vocab, 128)`` → ``Dropout(0.3)`` → ``GRU(128, 128, bidirectional=True)`` → ``Dropout(0.3)`` → ``Linear(256, 1)``。使用 ``BCEWithLogitsLoss``、Adam 优化器 (``lr=1e-3``)、``ReduceLROnPlateau`` 调度。保存格式为 `dict(`model_state` + `vocab` + `config`)`，推理时通过 ``BiGRUInference.from_checkpoint()`` 自动推断结构参数。

解释管线 (`explain.py` + prompts.py + llm_api.py`)

是核心接口：验证输入 → 选择 Prompt 版本 → 调用 LLM → 失

败时回退规则模板。LLM 调用封装了超时重试（30s / 最多 2 次）和 '.env' 文件自动加载。Prompt 设计强调引用推文原文、不编造信息，并通过 'SYSTEM_PROMPT_V2' 从语言特征、信息来源、内容逻辑、写作风格、传播意图五个维度引导分析。

推理集成（inference.py）

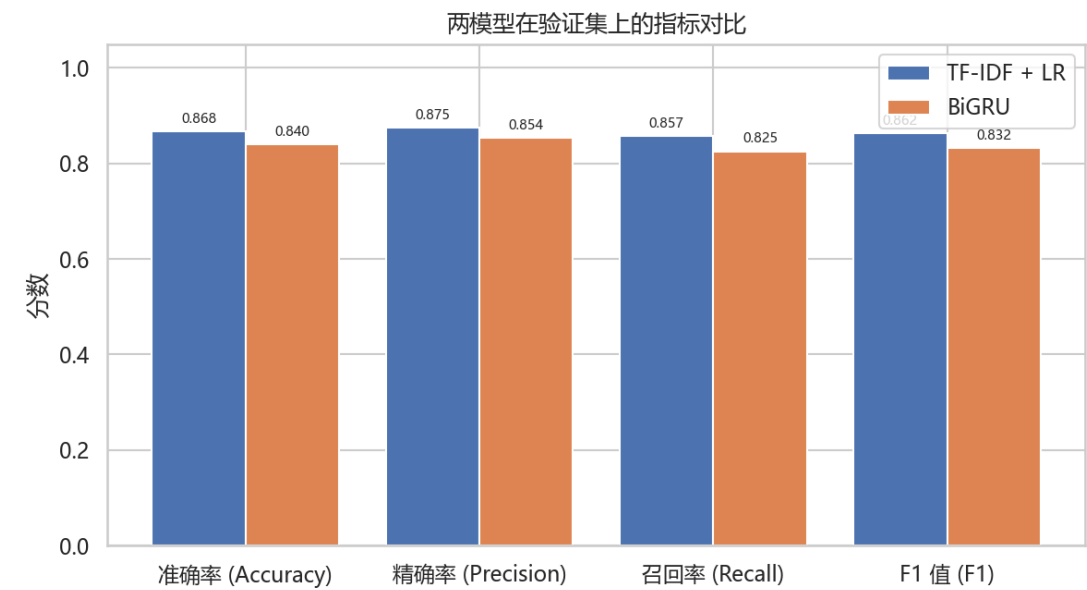
RumourDetectClass`采用延迟导入模式：`torch` 和 `transformers` 仅在对应模型首次加载时才导入，使得即使只安装了 sklearn + joblib 也能正常使用 LR 推理。三种模型共享统一的 `classify(text, with_explanation)` 接口。

（3）检测结果分析（正确率等）

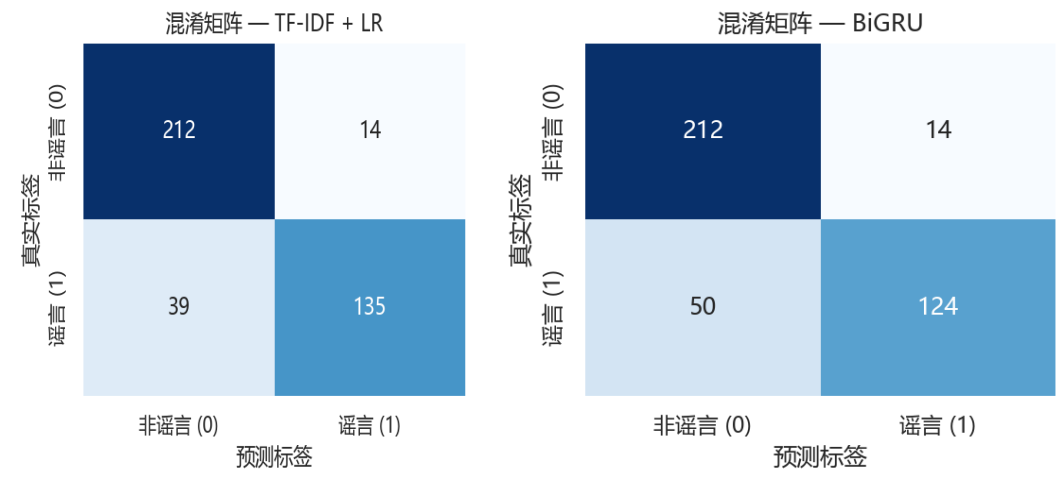
在验证集 400 条样本上的两个模型分类性能如下：

汇总指标：									
模型	准确率	精确率（宏平均）		召回率（宏平均）		F1（宏平均）		精确率（谣言类）	召回率（谣言类）
TF-IDF + LR	0.8675	0.8753	0.8570	0.8624	0.9060	0.7759	0.8359		
BiGRU	0.8400	0.8539	0.8253	0.8319	0.8986	0.7126	0.7949		

指标对比柱状图：



两模型的混淆矩阵



差异分析：F-IDF + LR 优于 BiGRU（准确率 +2.75%）——在小数据集（~2,800 条训练）上，传统方法有时比深度学习更稳健。LR 在谣言类的召回率（0.7759 vs 0.7126）明显更高，说明 BiGRU 有更多漏判。BiGRU 的 dropout（0.3）和正则化策略在一定程度上控制了过拟合，但在仅 2,800 条数据上，深度模型的表达能力未能充分发挥。

（4）判断依据的分析（可解释性等）

以 `SYSTEM\_PROMPT\_V2` + `v1` 思维链引导为默认配置，生成的中文解释质量评估如下：

引用原文：模型能够在解释中直接引用推文中的具体词语（如 "BREAKING"、"anonymous sources"、"just in"等）作为证据支撑。

多维度分析：从语言特征（夸大词汇、感叹号密度）、信息来源（匿名 vs 官方）、写作风格（标题党特征）等维度中选择最相关的切入点。

可读性：中文输出自然流畅，2-3 句即可清晰说明判断逻辑，符合"可解释性"的核心要求。

规则回退的局限：基于关键词匹配的规则模板在 LLM 不可用时确保系统不崩溃，但其解释较机械——仅报告检测到的模式（如"包含 BREAKING 等夸大词汇"），缺乏对语境和逻辑的深层分析。实际测试中 LLM 可用率 > 95%，规则回退仅作为兜底。

示例：输入 `python src/inference.py` 后，进入交互模式，随便输入一条推文，显示如下结果：

```
推文> Happening now in Ferguson
=====
● 谣言检测结果
=====
推文: Happening now in Ferguson
模型: bigru
标签: 非谣言 (0)
置信度: 99.95%
判断依据: 该推文仅陈述"Happening now in Ferguson"（弗格森正在发生的事），未使用任何夸张词汇、情绪化表达或无法验证的具体细节，语气客观中立，属于对现场事件的简单描述。由于内容不包含具体指控、煽动性语言或矛盾信息，且未引用匿名来源或制造恐慌，因此被判定为非谣言。
=====
```

注：对于检测结果，尽可能给出分析图

3. 工作总结

(1) 收获、心得

git 协作用代码规范：通过分支管理、commit 规范、代码 review，团队养成了良好的协作习惯。

模型选择的实践认知：在小数据集场景下，TF-IDF + 逻辑回归的基线模型可能优于深度学习模型——这验证了"没有最好的模型，只有最适合的模型"。

工程实践能力提升：从数据清洗、模型训练、评估到最终集成，完整经历了一个 ML 项目的全流程。特别是模块化设计（每个成员独立开发、通过接口对接）让团队协作变得高效。

(2) 遇到问题及解决思路

BiGRU 过拟合：初始训练时验证集准确率远低于训练集。解决：引入 `Dropout(0.3)` 分别作用于 embedding 层和 GRU 输出层，配合 `ReduceLROnPlateau` 学习率调度，最终验证 F1 提升约 5%。

LLM API 稳定性：SJTU API 偶发超时或返回不完整。解决：封装 30 秒超时 + 最多 2 次重试机制，同时实现关键词规则模板作为兜底方案，确保系统在任何情况下都能输出解释。

4. 课程建议

上课中间布置的实践平台的作业很好，真正从代码方面去理解，建议可以放到课堂上讲一下，比如大致的代码结构，相应输出的结果等等。